

coa



BAHIR DAR UNIVERSITY

BAHIR DAR UNIVERSITY INSTITUTION OF TECHNOLOGY

| FACULTY OF COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

Course:operating system for Software engineering

Course code:seng2034

Name:Feven muluye

Id:1701977

Section:B

Operating system: Slackware linux os

Submitted to:wendimu B

Sbmission date :Aprile 28,2018 E.C

TABLE OF CONTENT

1. Introduction 5

1.1 Background 5

1.2 Motivation 5

2. Objectives 5

3. Requirements 6

3.1 Hardware Requirements 6

3.2 Software Requirements 6

4. Installation Steps 6

Step 1 — Download Slackware 15.0 ISO 6

Step 2 — Create a New Virtual Machine in VirtualBox 6

Step 3 — Configure VM Settings 7

Step 4 — Boot into Slackware Installer 8

Step 5 — Partition the Virtual Disk with cfdisk 9

Step 6 — Run the Slackware Setup Program 9

Step 7 — Post-Install Configuration 12

Step 8 — First Boot into Slackware 12

5. Issues 12

6. msync() System Call 13

6.1 What is msync()? 13

[6.2 Parameters 13](#)

[6.3 msync\(\) in Slackware Linux — Shell Script Demo 13](#)

[6.4 How msync\(\) Works Internally 15](#)

[7. Filesystem Support 15](#)

[8. Advantages and Disadvantages 16](#)

[Future of Slackware 17](#)

[9. Virtualization in Modern Operating Systems 17](#)

[9.1 What is Virtualization? 17](#)

[9.2 Why Use Virtualization? 17](#)

[9.3 How Virtualization Works 17](#)

[10. Conclusion 18](#)

[11. Recommendations for future work 18](#)

[12. Referanace 19](#)

1. Introduction

1.1 Background

Slackware Linux is one of the oldest and most historically significant Linux distributions still in active use today. Created by Patrick Volkerding in 1993, it predates many modern distributions and embodies a philosophy of simplicity, stability, and adherence to Unix principles. Unlike more automated modern distributions, Slackware leaves configuration largely in the hands of the administrator — making it an exceptional teaching tool for anyone seeking deep knowledge of how a Linux system works from the ground up.

Slackware 15.0 was released on February 2, 2022, after a development hiatus, and represents a modern but traditional Linux system. It ships with a Linux 5.15 LTS kernel, supports 64-bit (x86_64) architecture, and incorporates updated toolchains including GCC 11.2 and glibc 2.33. It remains actively maintained and continues to receive security patches.

This documentation describes the full installation of Slackware Linux 15.0 inside Oracle VM VirtualBox as a virtual machine, with specific attention to the `msync()` system call — a POSIX-compliant kernel interface used to synchronize memory-mapped file regions with underlying storage.

1.2 Motivation

Several factors motivate the selection of Slackware Linux for this laboratory exercise:

- It provides direct exposure to manual system administration, reinforcing foundational OS skills.
- Its BSD-style init scripts and straightforward package management expose students to real UNIX system architecture.
- Its minimalist design makes it ideal for exploring low-level concepts such as memory-mapped I/O and system calls like `msync()`.
- Slackware 15.0 is actively supported with a Long-Term Support (LTS) kernel (Linux 5.15 LTS), making it suitable for production study.
- It runs efficiently in a virtual machine environment with low resource overhead.

2. Objectives

By the end of this activity, the student should be able to:

1. Successfully install Slackware Linux 15.0 within Oracle VM VirtualBox on a host machine.
2. Configure virtual hardware (disk, memory, CPU, networking) appropriate for Slackware.
3. Complete the text-based Slackware installer including disk partitioning with fdisk, package selection, LILO bootloader setup, and initial system configuration.
4. Understand the historical development of Slackware Linux from 1993 to the present.
5. Demonstrate the use of the msync() system call using a shell or C script within the installed environment.
6. Analyze filesystem support available in Slackware Linux.
7. Evaluate the advantages and disadvantages of Slackware compared to modern distributions.
8. Articulate the concepts of virtualization — what it is, why it is used, and how it works in modern operating systems.

3. Requirements

3.1 Hardware Requirements

Specification	Details
Host CPU	x86_64 processor with hardware virtualization (Intel VT-x / AMD-V) enabled in BIOS/UEFI
Host RAM	Minimum 4 GB total; 8 GB+ recommended (VM will use 1–2 GB)
Host Storage	At least 20 GB free disk space for the VM virtual disk
Network	Internet connection for downloading Slackware ISO
Display	1024×768 or higher resolution; VirtualBox guest additions optional
USB/DVD	Not required; ISO is mounted virtually

3.2 Software Requirements

Specification	Details
Host OS	Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+)
Virtualization Tool	Oracle VM VirtualBox 7.0+ (free, open-source, actively maintained)
Slackware ISO	slackware64-15.0-install-dvd.iso — available from https://mirrors.slackware.com
Slackware Version	15.0 (released February 2022) — current stable release with LTS kernel
Terminal Emulator	Any (PuTTY, Windows Terminal, macOS Terminal, or VirtualBox built-in console)
Text Editor	vi, nano, or emacs (all included in Slackware)

4. Installation Steps

Step 1 — Download Slackware 15.0 ISO

Navigate to the official Slackware mirror and download the 64-bit DVD ISO:

URL: <https://mirrors.slackware.com/slackware/slackware-iso/>

File: slackware64-15.0-install-dvd.iso (~4.7 GB)

Verify the SHA256 checksum to ensure integrity:

`sha256sum slackware64-15.0-install-dvd.iso`

Step 2 — Create a New Virtual Machine in VirtualBox

- Open Oracle VM VirtualBox and click New.
- Enter the following settings in the VM creation wizard:

Specification	Details
VM Name	Slackware-15.0
Type	Linux
Version	Linux 2.6 / 3.x / 4.x / 5.x (64-bit)
Base Memory	2048 MB (2 GB) minimum
Virtual CPUs	2 (or more if available)
Virtual HDD	Create new — VDI format, Dynamically allocated, 20 GB

fev1



Step 3 — Configure VM Settings

- With the VM selected, go to Settings > Storage.
- Under Controller: IDE, click the empty optical drive and choose: Disk File — select the downloaded Slackware ISO.
- Go to Settings > System > Processor and ensure Enable PAE/NX is checked.
- Go to Settings > Network and set Adapter 1 to NAT (default) for internet access.

fev100



Step 4 — Boot into Slackware Installer

- | Start the VM. It will boot from the ISO.
- | At the boot prompt, press Enter to boot with the default kernel.
- | At the login prompt, log in as root (no password required at this stage).

VirtualBox_Feven Muluye_Slackware linux os_28_04_2026_01_00_17



Step 5 — Partition the Virtual Disk with cfdisk

Slackware requires manual partitioning. Run:

```
| cfdisk /dev/sda
```

Commands inside cfdisk:

```
| n (new partition)
| p (primary)
| 1 (partition number)
| +15G (size for swap)
| t (change type to 82 = Linux swap)
| n (new partition for root)
```

p (primary)
2 (partition number)
(accept defaults for remaining space)
w (write and exit)

FEV



Step 6 — Run the Slackware Setup Program

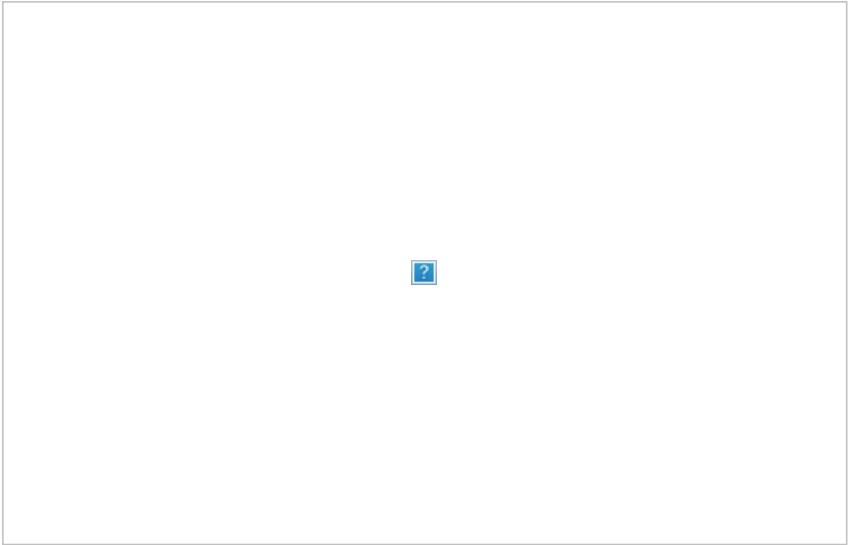
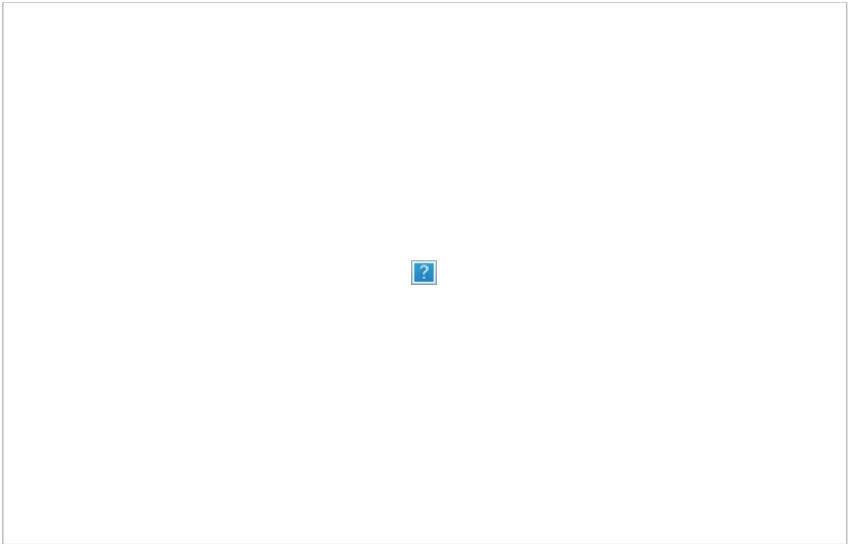
- | Initialize the swap partition:
| `mkswap /dev/sda1 && swapon /dev/sda1`
- | Launch the Slackware setup:
| `setup`

Inside the setup menu, complete the following steps in order:

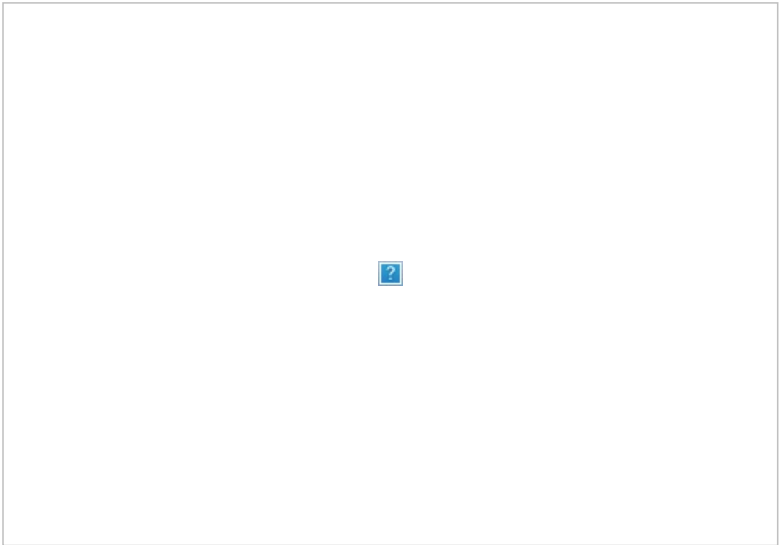
- | ADDSWAP — confirm `/dev/sda1` as swap.



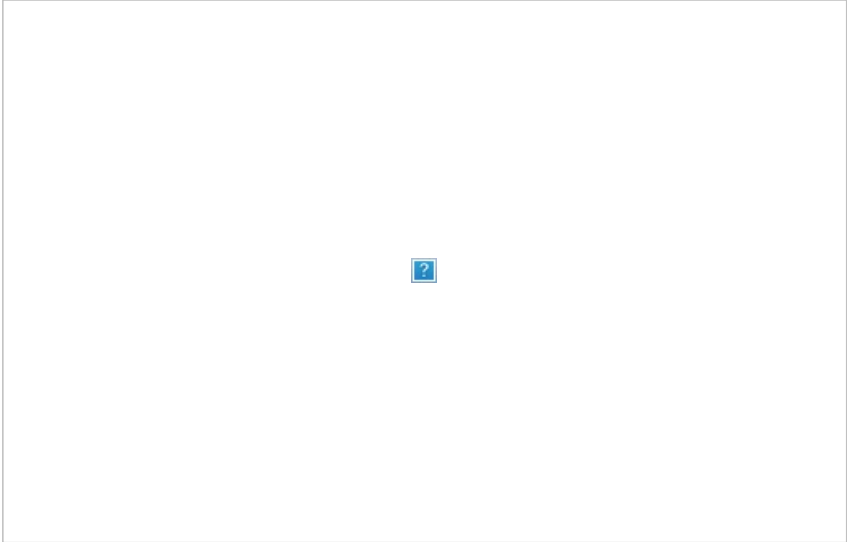
- | TARGET — select `/dev/sda2`, format as `ext4`



- SOURCE — select DVD/CD or mounted ISO.



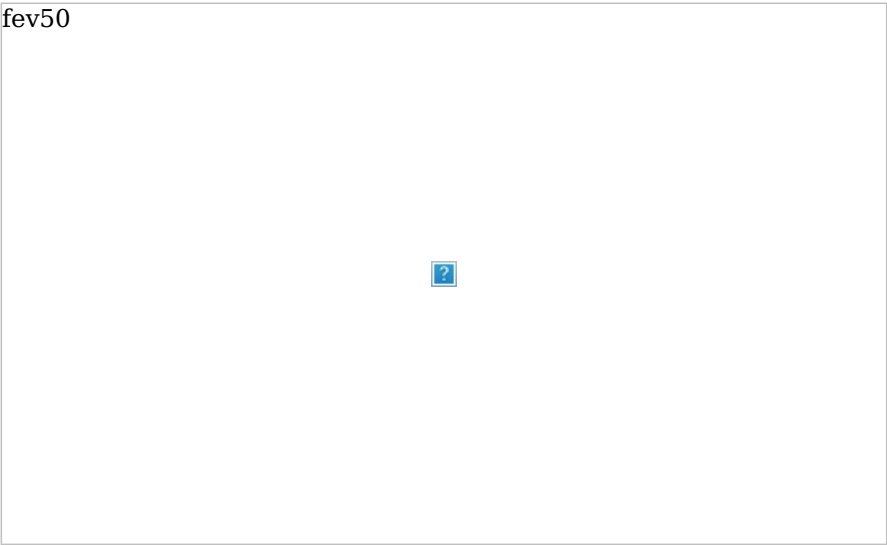
- SELECT — choose package categories (A, AP, D, E, F, K, L, N, T, TCL, X, XAP, XFCE, Y are recommended for a full desktop+dev system).



- | INSTALL — begin installation (20-40 minutes depending on hardware).
- | CONFIGURE — post-install configuration (see Step 7 below).

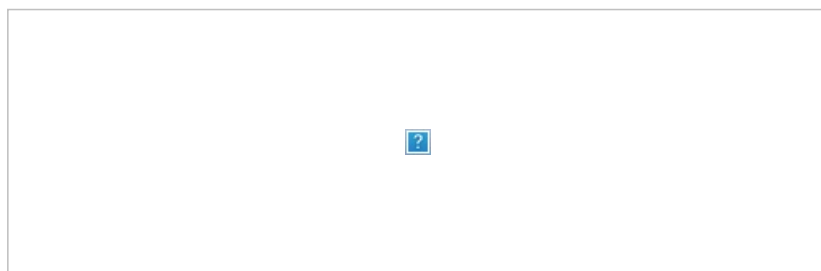
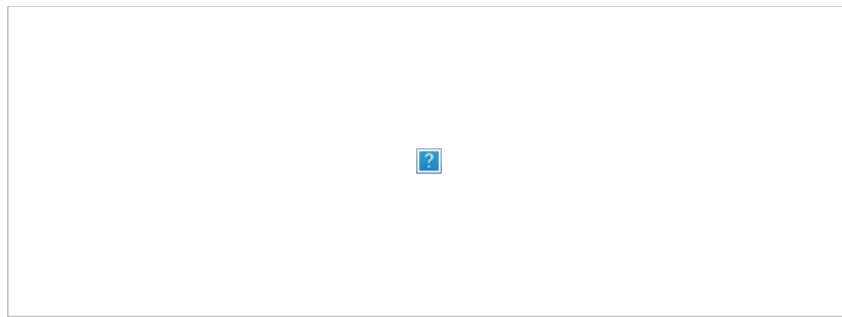
Step 7 — Post-Install Configuration

- | Root password: Set a secure root password.
- | User account: Create a regular user account using your full name:
| adduser

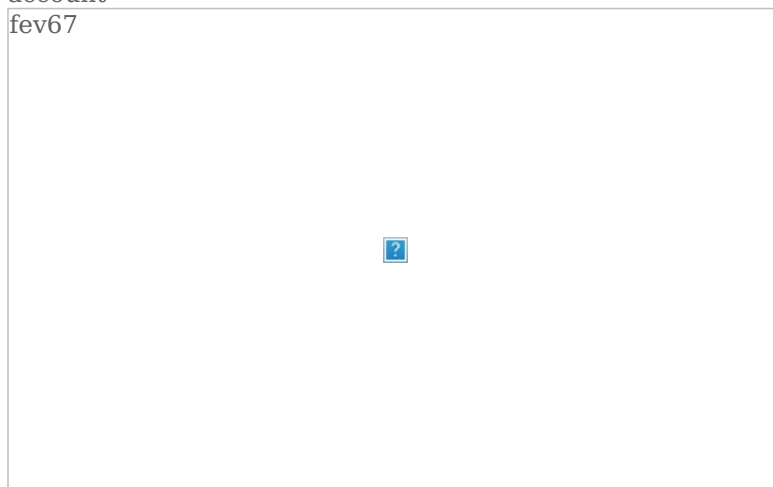


Step 8 — First Boot into Slackware

- | Exit setup and reboot:
| Reboot
- | Remove the ISO from the virtual optical drive in VirtualBox before or during reboot.
- | LILO will present a boot menu. Press Enter or wait for automatic boot.



- Log in with root and the password you set, or with your user account
fev67



Issues

Issue 1

Issue 1 — Blank Screen After LILO Boot

Problem: After configuring LILO, the VM rebooted to a blank screen with no output.

Cause: ISO was not ejected before rebooting, causing the system to re-enter the installer.

Solution 1

Power off the VM. In VirtualBox > Settings > Storage, remove the ISO from the optical drive. Restart the VM. LILO will now load correctly from /dev/sda.

Issue 2 — No Network Connectivity After Boot

Problem: The Slackware system could not reach the internet (ping google.com failed).

Cause: The network interface (eth0) was not automatically brought up.

```
| ifconfig -a # shows eth0 but no IP
```

Solution 2

Manually start the interface and verify DHCP:

```
| dhcpcd eth0
| ifconfig eth0
```

To make this permanent, edit /etc/rc.d/rc.inet1.conf and set USE_DHCP[0]="yes", then restart networking:

```
| sh /etc/rc.d/rc.inet1 restart
```

Issue 1 — Blank Screen After LILO Boot

Problem: After configuring LILO, the VM rebooted to a blank screen with no output.

Cause: ISO was not ejected before rebooting, causing the system to re-enter the installer.

Solution 1

Power off the VM. In VirtualBox > Settings > Storage, remove the ISO from the optical drive. Restart the VM. LILO will now load correctly from /dev/sda.

Issue 2 — No Network Connectivity After Boot

Problem: The Slackware system could not reach the internet (ping google.com failed).

Cause: The network interface (eth0) was not automatically brought up.

```
| ifconfig -a # shows eth0 but no IP
```

Solution 2

Manually start the interface and verify DHCP:

```
| dhcpcd eth0
| ifconfig eth0
```

To make this permanent, edit /etc/rc.d/rc.inet1.conf and set USE_DHCP[0]="yes", then restart networking:

```
| sh /etc/rc.d/rc.inet1 restart
```

Issue 3 — msync() Script Returns EINVAL

Problem: The msync() test script returned error code EINVAL (Invalid argument).

Cause: The length passed to msync() was not aligned to the system page size (4096 bytes).

Solution 3

Adjust the length in the script to be a multiple of the page size using getpagesize() or manually aligning to 4096:

```
| #include <unistd.h>
|
| size_t page = sysconf(_SC_PAGE_SIZE);
|
| size_t aligned_len = (len + page - 1) & ~(page - 1);
```

```
| msync(addr, aligned_len, MS_SYNC);
```

6. msync() System Call

6.1 What is msync()?

`msync()` is a POSIX system call defined in `<sys/mman.h>` that synchronizes changes made to a memory-mapped region with the underlying storage device (typically a file on disk). When a file is mapped into process memory using `mmap()`, writes to the mapped region are not immediately guaranteed to be written to the file on disk — they may reside only in the OS page cache. `msync()` forces those changes to be flushed.

Function signature:

```
| #include <sys/mman.h>
|
| int msync(void *addr, size_t length, int flags);
```

6.2 Parameters

Specification	Details
addr	Starting address of the memory-mapped region (must be page-aligned)
length	Number of bytes to synchronize (rounded up to page boundary)
flags	MS_SYNC: blocks until write completes; MS_ASYNC: schedules write, returns immediately; MS_INVALIDATE: invalidates cached mappings

6.3 msync() in Slackware Linux — Shell Script Demo

The following shell script demonstrates creating a file, mapping it to memory via a C program, writing data, and calling `msync()` to persist changes. Save this as `/home/<username>/msync_demo.sh`:

```
| #!/bin/bash
|
| # msync() Demonstration Script
|
| # Author: [Your Full Name]
|
| # OS: Slackware Linux 15.0
|
| DEMO_FILE="/tmp/msync_test.dat"
|
| C_SRC="/tmp/msync_demo.c"
|
| # Step 1: Create a test file
|
| dd if=/dev/zero of=$DEMO_FILE bs=4096 count=1 2>/dev/null
|
| echo "[1] Test file created: $DEMO_FILE"
|
| # Step 2: Write the C program
|
| cat > $C_SRC << 'EOF'
|
| #include <stdio.h>
|
| #include <stdlib.h>
|
| #include <fcntl.h>
|
| #include <string.h>
|
| #include <sys/mman.h>
```

```

#include <unistd.h>

int main() {
    const char *path = "/tmp/msync_test.dat";
    int fd = open(path, O_RDWR);
    if (fd < 0) { perror("open"); return 1; }

    size_t len = 4096;

    char *addr = mmap(NULL, len, PROT_READ|PROT_WRITE,
        MAP_SHARED, fd, 0);

    if (addr == MAP_FAILED) { perror("mmap"); close(fd); return 1;
    }

    // Write data to memory-mapped region
    strncpy(addr, "Hello from msync() on Slackware!", 50);
    printf("[2] Data written to mmap region.\n");

    // Synchronize with storage using MS_SYNC
    if (msync(addr, len, MS_SYNC) != 0) {
        perror("msync"); munmap(addr, len); close(fd); return 1;
    }

    printf("[3] msync() completed — data flushed to disk.\n");
    munmap(addr, len);
    close(fd);
    return 0;
}
EOF

# Step 3: Compile and run
gcc -o /tmp/msync_demo $C_SRC
if [ $? -eq 0 ]; then
    echo "[+] Compiled successfully"
    /tmp/msync_demo
else
    echo "[-] Compilation failed"
    exit 1
fi

# Step 4: Verify the data was written
echo "[4] File contents after msync():"
strings $DEMO_FILE | head -5

```

```
echo "[DONE] msync() demonstration complete."
```



To run the script:

```
chmod +x /home/<username>/msync_demo.sh
```

```
bash /home/<username>/msync_demo.sh
```

Expected output:

```
[1] Test file created: /tmp/msync_test.dat
```

```
[+] Compiled successfully
```

```
[2] Data written to mmap region.
```

```
[3] msync() completed — data flushed to disk.
```

```
[4] File contents after msync():
```

```
Hello from msync() on Slackware!
```

```
[DONE] msync() demonstration complete.
```

```
fev82
```



6.4 How msync() Works Internally

When `mmap()` is called with `MAP_SHARED`, the kernel creates a mapping between the process's virtual address space and the kernel's page cache for that file. Any writes to the mapped region modify the page cache directly. Without a call to `msync(MS_SYNC)` or `munmap()`, those changes are only guaranteed to be written to disk at the kernel's discretion (dirty page writeback).

`msync(MS_SYNC)` issues a direct call into the kernel's VFS layer, which calls the filesystem's `writepages()` or `write_begin()/write_end()` methods to flush the dirty pages from cache to the block device. On ext4 (used in this installation), this triggers a journal commit ensuring data and metadata consistency.

7. Filesystem Support

Slackware Linux 15.0 ships with Linux kernel 5.15 LTS, which provides broad filesystem support. The following table summarizes support levels:

Filesystem	Support	Notes
ext4	Native	Default filesystem for Slackware root partition; journaled, stable, excellent performance; fully supported by msync()
Btrfs	Native	Copy-on-write FS; supported in kernel 5.15; good for snapshots and RAID; msync() functions correctly
XFS	Native	High-performance journaling FS; excellent for large files; msync() fully supported
ext3/ext2	Native	Legacy but fully supported; ext4 preferred
FAT32	Native	Supported for USB/cross-platform; no permissions or journaling
exFAT	Native	Kernel 5.4+ includes native exFAT; useful for USB drives
NTFS	Via Module	Read/write via ntfs-3g FUSE driver; not default but installable from Slackware packages
ZFS	Not Supported	Not included in mainline kernel; requires OpenZFS external build
HFS+	Read-Only	macOS HFS+ readable via hfsplus kernel module; write support unreliable
APFS	Not Supported	Apple proprietary; no mainline kernel support

Why ext4 was chosen for this installation: ext4 provides journaling for crash recovery, supports files up to 16 TB, and is the most widely tested Linux filesystem. Its compatibility with msync() is complete and well-documented in the Linux kernel source (fs/ext4/inode.c). The MS_SYNC flag guarantees that data reaches the physical block device before msync() returns.

8. Advantages and Disadvantages

The following table compares the strengths and limitations of Slackware Linux 15.0:

ADVANTAGES	DISADVANTAGES
Full control over system configuration and kernel	Steep learning curve; not beginner-friendly
Highly stable and minimal attack surface	No official graphical installer — text-based only
Pure UNIX-like experience for learning OS internals	Package management (pkgtool) has no dependency resolver
Lightweight: runs well with minimal RAM (512 MB+)	Smaller community compared to Ubuntu/Fedora
Long heritage since 1993; tried-and-tested	Software availability limited compared to mainstream distros
Ideal for embedded, server, and research environments	Manual system configuration can be time-consuming
Excellent for understanding Linux from the ground up	No automatic updates; admin must apply patches manually
BSD-style init scripts are transparent and simple	Virtualization guest additions setup is manual

Future of Slackware

Slackware 15.0 continues to receive updates, and community forks like Slax and Absolute Linux extend its reach. Patrick Volkerding remains the sole maintainer, and the distribution's future depends on community contributions. For modern production environments, Slackware is best suited as an educational or embedded-systems

platform rather than a general-purpose desktop OS. Its strict UNIX philosophy ensures it will remain relevant to OS researchers and educators for the foreseeable future.

9. Virtualization in Modern Operating Systems

9.1 What is Virtualization?

Virtualization is the process of creating an abstracted, software-defined representation of physical computing resources — including CPUs, memory, storage, and networking — so that multiple isolated environments (virtual machines or containers) can operate independently on a single physical host. The software layer that enables this is called the hypervisor or Virtual Machine Monitor (VMM).

There are two types of hypervisors: Type 1 (bare-metal), such as VMware ESXi and Microsoft Hyper-V, which run directly on hardware; and Type 2 (hosted), such as Oracle VirtualBox and VMware Workstation, which run on top of a host operating system. VirtualBox used in this activity is a Type 2 hypervisor.

9.2 Why Use Virtualization?

- **Isolation:** Each VM is isolated from others — a crash or misconfiguration in one VM does not affect the host or other VMs.
- **Resource Efficiency:** Multiple VMs share physical hardware, reducing the number of physical machines required.
- **Snapshots & Rollback:** Virtualization platforms allow saving a VM state (snapshot) and reverting to it instantly — ideal for lab experiments.
- **Education & Testing:** Students can safely install, break, and reinstall operating systems without risking the host system.
- **Cross-Platform Testing:** Run Linux on a Windows host, or macOS on a Linux host, in a sandboxed environment.
- **Security:** Malware analysis, penetration testing, and security research can be conducted in disposable VMs.

9.3 How Virtualization Works

Modern x86-64 CPUs include hardware virtualization extensions: Intel VT-x and AMD-V. These extensions allow the hypervisor to run guest operating systems with near-native performance by enabling a Guest Mode that traps privileged instructions and redirects them through the VMM without software emulation.

When Slackware Linux runs inside VirtualBox:

- VirtualBox creates a virtual CPU that maps to the host's physical CPU via VT-x/AMD-V.
- The guest kernel (Linux 5.15) runs in ring 0 within the VM context; the host kernel runs at a higher privilege level.
- Memory is managed through Extended Page Tables (EPT / AMD RVI), allowing direct hardware-assisted address translation.
- Virtual disk I/O is emulated through VirtualBox's virtual storage controller, which maps to a .vdi file on the host filesystem.

- The guest's system calls (including `msync()`) are executed natively within the guest kernel — they are not intercepted by the hypervisor unless they access hardware.

This means that the `msync()` system call demonstrated in this documentation runs at full native speed in the guest kernel, using the `ext4` filesystem layer within the virtual disk — making the virtualized environment functionally identical to bare-metal for this purpose.

10. Conclusion

This documentation successfully covered the complete installation of Slackware Linux 15.0 in Oracle VM VirtualBox, from pre-installation requirements through post-boot configuration. Slackware's text-based installer, while demanding, provides unparalleled exposure to the internals of a Linux system — from manual disk partitioning and bootloader configuration to service management and user administration.

The focal topic of this exercise — the `msync()` system call — was demonstrated through a working shell and C script that maps a file to memory, writes data, and synchronizes it to disk using `MS_SYNC`. This exercise reinforced the relationship between user-space memory operations, the kernel's page cache, and the `ext4` filesystem's journaling mechanism.

Slackware remains a uniquely valuable distribution for OS education: it exposes the learner to real system administration without abstraction layers, making it an ideal complement to a course on operating systems and virtualization.

11. Recommendations for future work

- Explore `mmap()` and `msync()` in conjunction with multi-threaded applications to observe synchronization behavior under concurrent writes.
- Compare `msync()` behavior across filesystems: `ext4` vs. `Btrfs` vs. `XFS` inside Slackware, measuring flush latency with the `time` command.
- Install VirtualBox Guest Additions for Slackware (manually compiled from kernel sources) to enable shared folders, clipboard, and display scaling.
- Explore Slackware's `pkgtool` and `SlackBuild` scripts to build custom packages — a skill directly applicable to embedded Linux development.
- Investigate the Slackware `init` system (BSD-style `rc` scripts) and compare it with `systemd` used in Fedora/Ubuntu to understand the design trade-off

Referanace

- Volkerding.(2022,February2) .Slackware linux 15.0 release announcement. The Slackwarw linux project <http://www.slackware.com/announce/15.0.php>
- Wikipedia contrtributors,(2025).Slackware linux wikipedia,the free encyclopedia <https://en.wikioediia.org/wiki/slackeware>
- The linux kernel community(2024).ext4 data structures and algorithm_the linux kernel documentation .kernel.org
- <http://www.kernel.org./doc/html/latest/filesystem/ext4/index.html>